

Introduction to Parallel Algorithms and Programming

Session 5: Introduction to GPU programming with CUDA

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- What a **GPU** is, how it is built, and where it sits next to the CPU
- The general principle of every GPU program: **allocate, copy in, launch N threads, copy out**
- The minimal CUDA toolbox, one item at a time:
 - memory: `cudaMalloc`, `cudaMemcpy`, `cudaFree`
 - threads: `__global__`, `f<<<B, T>>>()`, `cudaDeviceSynchronize`
 - who am I: `blockIdx.x * blockDim.x + threadIdx.x`, `gridDim.x * blockDim.x`
- Two complete programs: a hello world with thread ids, and `MULTIPLYBYTWO` step by step
- The dictionary PRAM / OpenMP / CUDA: the same ideas, a third syntax; how to compile and run
- Not today: shared memory, `__syncthreads`, atomics, warps, memory coalescing, ...: the GPU programming course (M2)

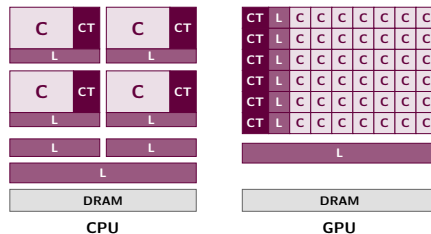
Outline

- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: `MULTIPLYBYTWO`, step by step
- 6 Summary
- 7 Compiling and running; the lab

- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: MULTIPLYBYTWO, step by step
- 6 Summary
- 7 Compiling and running; the lab

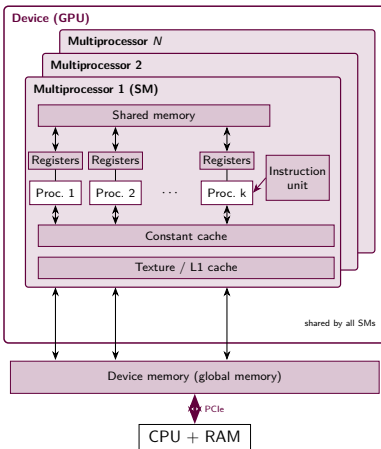
What is a GPU?

- A **second processor** on its own card, built to draw pictures: the **same operation on millions of pixels**
- In 2006 it was **“hacked”** into general computing: Ian Buck’s matrix product, disguised as a picture. The idea behind **CUDA** (2007), the GPU programmable in C
- **Thousands of small cores** rather than a few big ones; no large caches, no branch prediction, no out-of-order execution: the chip area goes to arithmetic
- Built for **data parallelism**, the **forall** of every PRAM algorithm
- **Its own memory** (up to a few hundred GB, several TB/s), **separate** from the CPU’s
- Today: NVIDIA GPUs and CUDA, C++ with a few additions, compiled by `nvcc`



C: core, CT: control circuitry, L: cache. The same chip area: a few big cores with large caches and control on the CPU, a sea of small cores on the GPU.

Inside a GPU, and where it sits next to the CPU



- A GPU is made of N **streaming multiprocessors** (SMs), a few dozen to 150. Each SM is a small SIMD processor: up to **1024 logical CUDA cores** (typically 64 to 128 physical ones), one instruction unit driving them, registers and small memories
- A core runs **one thread** at a time; thousands of threads live on the SMs at once, and an SM switches between them for free: this hides the latency of the memory
- All the SMs share the **global memory** of the GPU
- The GPU is a **coprocessor**, the CPU is in charge: it allocates GPU memory, copies data in, launches work, waits, copies results out
- The two memories talk over the **PCIe** bus (up to 128 GB/s with PCIe 6.0), far slower than either memory: transfers are the expensive part

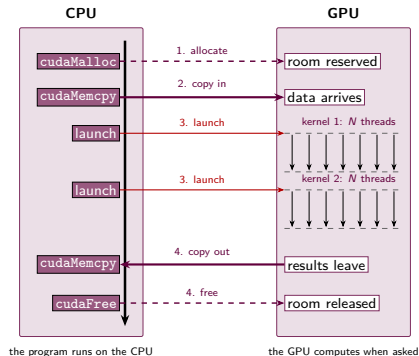
- 1 What is a GPU?
- 2 How a GPU program works**
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: MULTIPLYBYTWO, step by step
- 6 Summary
- 7 Compiling and running; the lab

The general principle of a GPU program

The data and the program start on the CPU, in its memory. The GPU **cannot read it**: it needs the data in **its own memory**. Hence the same four steps in every GPU program.

- 1 **Allocate** room in the GPU memory (`cudaMalloc`)
- 2 **Copy the data in**, CPU → GPU (`cudaMemcpy`), before any computation starts on the GPU
- 3 **Launch N GPU threads**, all running the same function, the **kernel**, each with a different index. The index distributes the work: every element handled by one thread, none forgotten
- 4 **Copy the results out**, GPU → CPU (`cudaMemcpy`); free the GPU memory (`cudaFree`)

The CPU program drives everything, in this order. Step 3 is the PRAM-like algorithm (several kernels may follow one another); steps 1, 2 and 4 are its shipping cost.

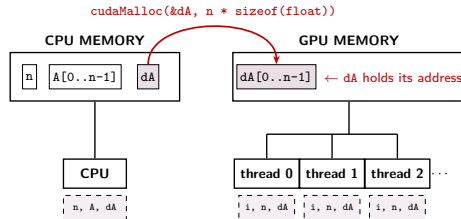


- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers**
- 4 Threads: launching them, and who am I
- 5 A complete example: MULTIPLYBYTWO, step by step
- 6 Summary
- 7 Compiling and running; the lab

How to allocate memory on the GPU

```
int n = 10; float c = 2; // scalars
float A[10] = {...};    // a CPU array
float *dA;               // for a GPU address
cudaMalloc(&dA, n * sizeof(float));
...
f<<<B, T>>>(dA, n, c);  // address + values
...
cudaFree(dA);           // when done
```

- **Scalars** (a size n , a factor c): just **pass them as kernel arguments**, by value: one copy per thread, BCAST for free
- **Arrays**: the GPU needs **its own copy**.
`cudaMalloc(&dA, size)` reserves `size` bytes in the GPU memory and stores their address in `dA`; `cudaFree(dA)` releases them:
`malloc/free`, on the other memory



- `dA` is a **CPU variable holding a GPU address** (`d` for *device*, the GPU): the CPU passes it to kernels, **never** reads `dA[i]`; a kernel reads `dA[i]`, **never** a CPU array
- What threads **share** is a GPU array; what a thread declares **inside the kernel** is private: the two rules of session 3

How to transfer data between the CPU and the GPU

- **One function, both directions:**

`cudaMemcpy(dst, src, size, direction)`, a memcpy with a direction. *Host* is the CPU, *device* the GPU

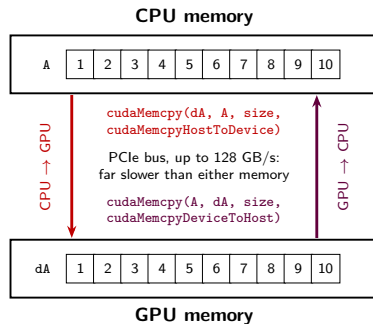
- Sizes in **bytes**: `n * sizeof(float)`, like `malloc`

- **Synchronous**: it returns when the copy is done, and it **waits for the kernels launched before it**: the results come back complete

- no need to call `cudaDeviceSynchronize()` before a `cudaMemcpy`

- Through the PCIe bus: **slow**. Ship the data once, run as many kernels as needed, bring the results back once

```
// CPU -> GPU, before the kernels
cudaMemcpy(dA, A, n * sizeof(float),
           cudaMemcpyHostToDevice);
... kernels working on dA ...
// GPU -> CPU: the results come back into A
cudaMemcpy(A, dA, n * sizeof(float),
           cudaMemcpyDeviceToHost);
```



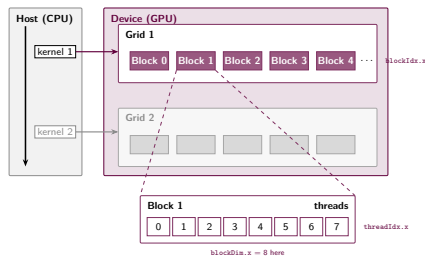
- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I**
- 5 A complete example: `MULTIPLYBYTWO`, step by step
- 6 Summary
- 7 Compiling and running; the lab

How to run N GPU threads: blocks and threads

- A **kernel** is a function marked `__global__`, returning void: the program of **one thread** (like the body of the **forall** in PRAM)
- **Two levels** on the GPU (SMs, cores), two levels in a launch: `f<<<B, T>>>` creates **B blocks of T threads**; each **block** goes to one **SM**, its **T threads** to the **SM's cores**
- $B \times T$ threads in total (the *grid*, figure): 8 threads as `<<<1, 8>>>`, `<<<2, 4>>>`, `<<<4, 2>>>` or `<<<8, 1>>>`. Limit: $T \leq 1024$, B practically unlimited. Rule of thumb: $T = 256$, $B = \lceil N/T \rceil$
- The launch returns **at once**: `cudaDeviceSynchronize()` makes the CPU wait for every thread

```
__global__ void f(float *dA, int n) // the kernel
{
    ... // the program of one thread
}

...
f<<<B, T>>>(dA, n); // B blocks of T
cudaDeviceSynchronize(); // wait
```



Who am I? Global thread index and number of threads

- Four variables **defined automatically inside a kernel**:

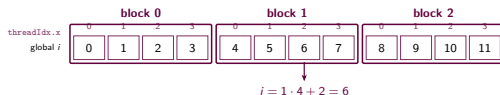
- `threadIdx.x`: my number inside my block, from 0 to $T - 1$
- `blockIdx.x`: the number of my block, from 0 to $B - 1$
- `blockDim.x`: the number of threads per block, T
- `gridDim.x`: the number of blocks, B

- The one formula: $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$, unique, from 0 to $BT - 1$: the **index i of the PU** (from 0), the `omp_get_thread_num()` of session 3

- $p = \text{gridDim.x} * \text{blockDim.x}$: the number of threads, the P of the PRAM, `omp_get_num_threads()`

- Every kernel starts with these two lines; then thread i works on element i

```
__global__ void f(float *dA, int n)
{
    int i = blockIdx.x * blockDim.x
          + threadIdx.x;          // my index
    int p = gridDim.x * blockDim.x; // # threads
    ...
}
```



Three blocks of four: `threadIdx.x` above each cell, the global index i inside. Thread 2 of block 1 is $i = 1 \cdot 4 + 2 = 6$.

Hello world with thread ids (02-hello-ids.cu)

```
1 #include <stdio>
2 #include <cuda_runtime.h>
3
4 __global__ void hello()
5 {
6     int i = blockIdx.x * blockDim.x + threadIdx.x; // my index
7     int p = gridDim.x * blockDim.x;                // how many of us
8     printf("I am thread %d of %d\n", i, p);
9 }
10
11 int main()
12 {
13     hello<<<2, 4>>>>(); // 2 blocks of 4 threads: 8 threads
14     cudaDeviceSynchronize();
15     return 0;
16 }
```

- Line 2: `cuda_runtime.h` provides all the CUDA functions, as `omp.h` does for OpenMP
- Line 13: `<<<2, 4>>>`, 2 blocks of 4 threads: **8 threads** run lines 6–8, each with its own `i` and `p`
- Output: 8 lines `I am thread i of 8, i = 0...7`, in a **different order** at every run: threads are asynchronous
- Line 14: without `cudaDeviceSynchronize()`, `main` returns while the GPU is still at work: nothing is printed
- `<<<4, 2>>>` or `<<<1, 8>>>`: the same eight `i`, packaged differently. The lab prints the four variables behind `i` and `p`

- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: MULTIPLYBYTWO, step by step**
- 6 Summary
- 7 Compiling and running; the lab

MULTIPLYBYTWO: the PRAM algorithm, and its kernel

MULTIPLYBYTWO($A[1 \dots N]$)

```
1: forall  $i = 1, \dots, N$  in parallel do
2:    $A[i] \leftarrow 2 \cdot A[i]$ 
```

```
5 __global__ void multiplyByTwo(float *dA, int n)
6 {
7   int i = blockIdx.x * blockDim.x + threadIdx.x;
8   if (i < n)           // beyond n: nothing
9     dA[i] = 2 * dA[i]; // my own cell
10 }
```

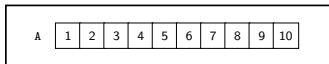
- The simplest PRAM algorithm: one step, N PUs, PU i reads and writes **its own cell**. Here $N = 10$ floats, and the array starts on the CPU
- The kernel is the **body of the forall**: line 7 computes the index, line 9 is the PRAM line, on the **GPU array** dA (the kernel's name for the address it receives)
- Line 8, the **guard** if ($i < n$): the " $i = 1, \dots, N$ " of the **forall**, for the threads whose index is beyond the array (see the launch step)
- n travels as an argument: a scalar, one copy per thread

Step 1: the data on the CPU, and room for it on the GPU

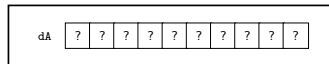
```
12 int main()
13 {
14     float A[N];
15     for (int i = 0; i < N; i++) A[i] = i + 1;
16     float *dA;           // GPU address
17     cudaMalloc(&dA, N * sizeof(float));
```

- Lines 14–15: the problem starts on the CPU, as always: A is a CPU array, filled by the CPU
- Line 16: dA, a CPU variable for a GPU address, empty so far
- Line 17: cudaMalloc reserves 10×4 bytes in the GPU memory and stores their address in dA:
allocated, not **filled**

CPU memory



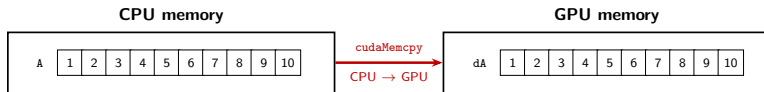
GPU memory



Step 2: copy the data in

```
18  cudaMemcpy(dA, A, N * sizeof(float),  
19          cudaMemcpyHostToDevice);
```

- Lines 18–19: the 40 bytes of A travel over the bus into dA; cudaMemcpyHostToDevice gives the direction
- From now on, **two copies** of the array exist, one in each memory. The GPU works on dA; A waits for the results
- The first of the two slow operations of the program



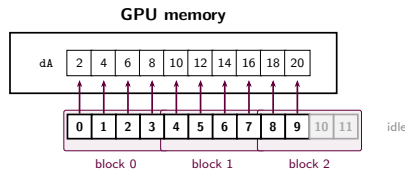
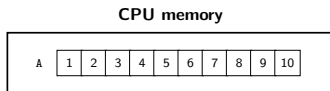
Step 3: launch the threads, one per cell (and a few spare)

```

21  int T = 4;                // threads per block
22  int B = (N + T - 1) / T;  // = 3
23  multiplyByTwo<<<B, T>>>(dA, N);

```

- $T = 4$ threads per block, $B = \lceil N/T \rceil = 3$ blocks: $(N + T - 1) / T$ rounds **up**, so that $BT \geq N$ and every cell gets a thread
- Line 23: 12 threads, $i = 0 \dots 11$; thread i doubles $dA[i]$: the **forall**, with the kernel as its body. The CPU does not wait: the launch returns at once
- Threads 10 and 11 have no cell: the guard `if (i < n)` makes them do **nothing**; without it they would write past the end (corrupted memory, no error)

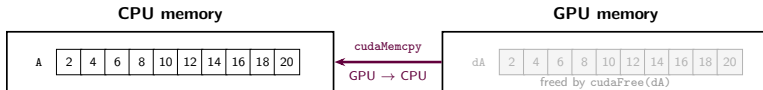


thread i : $dA[i] = 2 * dA[i]$ if $i < n$

Step 4: copy the results back, free the GPU memory

```
25  cudaMemcpy(A, dA, N * sizeof(float),  
26      cudaMemcpyDeviceToHost);  
27  cudaFree(dA);  
28  for (int i = 0; i < N; i++) printf("%g ", A[i]);  
29  printf("\n");
```

- Lines 25–26: cudaMemcpy **waits for the kernel**, then brings the 40 bytes of dA back into A: the second slow operation
- Line 27: cudaFree releases the GPU memory. Lines 28–29, on the CPU: 2 4 6 8 10 12 14 16 18 20
- With $N = 10^8$: the kernel takes milliseconds, each copy tens of milliseconds. Ship once, launch many kernels, bring back once



The whole program (multiply-by-two-steps.cu)

```
1 #include <stdio>
2 #include <cuda_runtime.h>
3 #define N 10
4
5 __global__ void multiplyByTwo(float *dA, int n)
6 {
7     int i = blockIdx.x * blockDim.x + threadIdx.x;
8     if (i < n)          // beyond n: nothing
9         dA[i] = 2 * dA[i]; // my own cell
10 }
```

- Four steps, four CUDA calls around the PRAM-like algorithm: `cudaMalloc` (17), `cudaMemcpy` in (18), the launch (23), `cudaMemcpy` out (25)
- Every data-parallel PRAM algorithm becomes a program this way: a kernel per step, on $\lceil N/T \rceil$ blocks of T threads, with the index and the guard

```
12 int main()
13 {
14     float A[N];
15     for (int i = 0; i < N; i++) A[i] = i + 1;
16     float *dA;          // GPU address
17     cudaMalloc(&dA, N * sizeof(float));
18     cudaMemcpy(dA, A, N * sizeof(float),
19               cudaMemcpyHostToDevice);
20
21     int T = 4;           // threads per block
22     int B = (N + T - 1) / T; // = 3
23     multiplyByTwo<<<B, T>>>>(dA, N);
24
25     cudaMemcpy(A, dA, N * sizeof(float),
26               cudaMemcpyDeviceToHost);
27     cudaFree(dA);
28     for (int i = 0; i < N; i++) printf("%g ", A[i]);
29     printf("\n");
30     return 0;
31 }
```

- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: MULTIPLYBYTWO, step by step
- 6 Summary**
- 7 Compiling and running; the lab

Summary: the dictionary PRAM / OpenMP / CUDA

PRAM	OpenMP (session 3)	CUDA (today)
forall $i = 1, \dots, N$ in parallel do <i>body</i> N PUs	#pragma omp parallel { <i>body</i> } a handful of threads (one per core)	kernel __global__ void f(...) { <i>body</i> }, launched by f<<<B, T>>>(...) $B \times T$ threads, $BT \geq N$, with the guard if (i < n)
index i of the PU number of PUs P the shared memory	omp_get_thread_num() omp_get_num_threads() shared variables (declared outside)	blockIdx.x * blockDim.x + threadIdx.x gridDim.x * blockDim.x GPU arrays (cudaMalloc, cudaMemcpy in/out, cudaFree), passed to the kernel as pointers
private registers	private variables (declared inside)	variables declared inside the kernel
BCAST(x, N) end of a step (lockstep)	reading a shared scalar #pragma omp barrier	a scalar kernel argument, copied to every thread none inside a kernel; the threads synchronize at its end (one kernel per step). cudaDeviceSynchronize() syncs the CPU with the GPU, not the threads
CRCW fusion write, or CREW + REDUCE	two threads writing one variable: a race ; own variable, then reduce	the same race : own cell, then reduce. CREW-like: reading one cell at once is fine

- 1 What is a GPU?
- 2 How a GPU program works
- 3 Memory: allocation and transfers
- 4 Threads: launching them, and who am I
- 5 A complete example: `MULTIPLYBYTWO`, step by step
- 6 Summary
- 7 Compiling and running; the lab**

Compiling and running a CUDA program

- Source files have the extension **.cu**: one file holds the CPU part (main) and the GPU part (the kernels)
- Compile with **nvcc**, NVIDIA's compiler, then run as usual:

```
nvcc -O2 prog.cu -o prog then ./prog
```

nvcc compiles the kernels for the GPU and hands the rest of the file to the C++ compiler (g++)

- `-arch=native` compiles for the GPU of the machine; `nvidia-smi` tells you which GPU you have, and whether its driver is installed
- No NVIDIA GPU in your laptop (none in any Mac)? **Compiler Explorer**, <https://cuda.godbolt.org>, compiles and runs a .cu file on a real GPU: language *CUDA C++*, compiler *NVCC*, add an executor, paste the file. Every program of today is a single file

Lab 3: Introduction to GPU programming with CUDA

Compiler Explorer (<https://cuda.godbolt.org>)

CUDA C++, NVCC, an executor with `-O2`. The GPU arrays and the copies are given in the skeletons; the constructs of today are the only ones you need. Write your answers as comments at the top of the files: the two `.cu` files are what you submit.

Ex. 1: hello (`hello.cu`)

- The kernel: every thread prints thread t/T of block b/B , global thread i/p
- 64 threads as 64×1 , 32×2 , $\dots$, 1×64 : which of the six values change from one configuration to the next, which do not?

Ex. 2: saxpy (`saxpy.cu`), $y \leftarrow a \cdot x + y$, $N = 10^6$

- The kernel: one element per thread, with the guard
- $T = 256$, $B = \lceil N/T \rceil$, the launch; the skeleton times the kernel and checks the result
- $T = 1, 2, 4, \dots, 1024$: still correct? how does the time change?
- Time the two copies as well: how do they compare with the kernel? What does it mean for a program with one kernel, with many?

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Paris, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com